NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

ATTRIBUTE GRAMMARS

IN OCAMLFLAT/OFLAT

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
February, 2025

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

ATTRIBUTE GRAMMARS

IN OCAMLFLAT/OFLAT

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

Adviser: Artur Miguel Dias
Professor Auxiliar, DI-FCT, Universidade NOVA de Lisboa

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
February, 2025

Attribute Grammars
in OCamlFLAT/OFLAT

Copyright © Pedro Emanuel Inácio Bailão, NOVA School of Science and Technology, NOVA University Lisbon.
The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical boundaries, to file and publish this dissertation through printed copies reproduced on paper or on digital form, or by any other means known or that may be invented, and to disseminate through scientific repositories and admit its copying and distribution for non-commercial, educational or research purposes, as long as credit is given to the author and editor.

This document was created with the (pdf/Xe/Lua)LaTeX processor and the NOVAthesis template (v7.1.29) [7].

Abstract

Theoretical concepts in Formal Languages and Automata Theory (FLAT) are known for their formal, rigorous, and often complex nature, making them challenging to learn. To address this challenge, various tools have been developed over the years, allowing users to engage with these concepts through practical exercises and examples.

At FCT-UNL, two pedagogical tools were started to aid in this learning process: the OCamlFLAT library, which provides logical support for FLAT concepts, and the OFLAT graphical application, a web-based tool offering interactive visualizations of these ideas.

The first goal of this dissertation is to extend the OCamlFLAT library by incorporating support for attribute grammars. The second goal is to enhance the OFLAT graphical application by adding features that allow for the visualization and interaction with these grammar concepts.

A key focus of this work is the pedagogical enhancement of the developed software. This dissertation explains some theoretical concepts of Theory of computation, emphasizing the topic of atribute grammars, discusses some of the best existing software aplications for Theory of computation concepts, describes the current state of OCamlFLAT/OFLAT, presents the main tools that are going to be used in the project and finishes with a work plan.

Keywords: Formal Languages, Automata Theory, OCaml, OFLAT, OCamlFLAT, Functional Programming, Formal Grammars

Resumo

Os conceitos teóricos em Teoria de Linguagens Formais e Autómatos (FLAT) são conhecidos pela sua natureza formal, rigorosa e muitas vezes complexa, o que os torna desafiantes para aprender. Para mitigar essa dificuldade, várias ferramentas foram desenvolvidas ao longo dos anos, permitindo que os utilizadores interajam com esses conceitos através de exercícios e exemplos práticos.

Na FCT-UNL, foram criadas duas ferramentas pedagógicas com este propósito: a biblioteca OCamlFLAT, que fornece suporte lógico para os conceitos de FLAT, e a aplicação gráfica OFLAT, uma ferramenta web que oferece visualizações interativas desses conceitos.

O primeiro objetivo desta dissertação é estender a biblioteca OCamlFLAT, incorporando suporte para gramáticas com atributos. O segundo objetivo é melhorar a aplicação gráfica OFLAT, adicionando funcionalidades que permitam a visualização e interatividade com estes conceitos de gramáticas.

o foco principal deste trabalho é a melhoria pedagógica do software desenvolvido.

Esta dissertação explica alguns conceitos teóricos da Teoria da Computação, com ênfase no tema das gramáticas com atributos, discute algumas das melhores aplicações de software existentes para conceitos da Teoria da Computação, descreve o estado atual do OCamlFLAT/OFLAT, apresenta as principais ferramentas que serão utilizadas no projeto e finaliza com um plano de trabalho.

Palavras-chave: Teoria de Linguagens Formais, Autómatos, OCaml, OFLAT, OCamlFLAT, Programação Funcional, Gramáticas Formais

Contents

List of Figures **ix**

1 Introduction **1**
1.1 Context 1
1.2 Objectives 1
1.3 Document Structure 1

2 Theory of Computation **3**
2.1 Chomsky Hierarchy 3
2.1.1 Type 0: Unrestricted Grammars 3
2.1.2 Type 1: Monotonic Grammars 4
2.1.3 Type 2: Context-Free Grammars 5
2.1.4 Type 3: Regular Grammars 5
2.2 Attribute Grammars 6
2.2.1 Synthesized Attributes 6
2.2.2 Inherited Attributes 7
2.2.3 Conditions 8
2.2.4 Circularity 9
2.3 Automata 11
2.3.1 Types of Automata 11

3 Grammar Tools **13**
3.1 JFLAT 13
3.2 ANTLR 14
3.3 CGM: Context-Free Grammar Manipulator 14

4 OFLAT/OCaml-Flat **15**
4.1 OCamlFLAT Overview 15
4.2 OFLAT User Interface 16

5 Technology **19**
5.1 Programming with OCaml 19
5.1.1 Declarative Programming 20
5.2 OCaml/JavaScript Interoperability 20
5.2.1 JavaScript Types 20
5.2.2 Bindings 21
5.2.3 `js_of_ocaml` Modules 21
5.3 Cytoscape.js 21

6 Planning **23**
6.1 Introduction 23
6.2 Support for Attribute Grammars in the OCamlFLAT Library (8 weeks) . 23
6.3 Support for Grammars in the OFLAT Web Application (6 weeks) 24
6.4 Pedagogical Exercises (1 week) 24
6.5 Evaluation (1 week) 24
6.6 Dissertation Writing (8 weeks) 24
6.7 Gantt Chart 25

Bibliography **27**

List of Figures

2.1 Chomsky hierarchy 4
2.2 Attribute grammar for signed binary numbers [4] 10
2.3 Attribute dependency graph for the grammar in figure 2 [4] 10
2.4 Modified attribute dependency graph with circularity 11

4.1 Current Version OFLAT User Interface 15
4.2 OFLAT User Interface Division. Red - Sidebar, Blue - Workspace 16
4.3 Workspace Closeup 17
4.4 Workspace Window Split 17

1

Introduction

1.1 Context

The study of Formal Languages and Automata Theory (FLAT) is integral to Computer Engineering and Computer Science programs. However, due to its abstract and mathematical nature, students often struggle to fully grasp the concepts and models involved. Therefore, a tool that complements the study of FLAT theory by being practical, user-friendly, and visually oriented is highly beneficial.

In recent years, the OCamlFLAT/OFLAT tool has been developed at FCT-UNL, offering extensive support in this field. Despite its advanced state, with support for several key models, there are still many features that could enhance its functionality.

This dissertation aims to introduce new capabilities to OCamlFLAT/OFLAT, enhancing both the OCamlFLAT library and the OFLAT graphical interface.

1.2 Objectives

The primary objective of this dissertation is to incorporate support for attribute grammars. Attribute grammars extend context-free grammars by allowing additional information (attributes) to be associated with production rules. These attributes are crucial for describing the semantic properties of the languages generated by the grammars.

To achieve this, the OCamlFLAT library needs to be extended to support attribute grammars. Additionally, graphical enhancements are required to enable the creation and editing of these grammars within the OFLAT platform.

1.3 Document Structure

This thesis is structured into six chapters:

- **Introduction**: Provides an overview of the dissertation's context, objectives, and structure.

- **Theory of Computation**: Lays the theoretical foundation for the study of FLAT models. It begins with an exploration of the Chomsky hierarchy and progresses into discussions of the four language types (Type 0, 1, 2, and 4).
- **Grammar Tools**: Examines existing pedagogical tools that support the teaching and learning of FLAT theory, with a focus on three prominent systems: JFLAT, ANTLR, and CGM.
- **OCamlFLAT and OFLAT Systems**: Introduces the OCamlFLAT library and the OFLAT platform.
- **Technology**: Covers the technical aspects of developing the OCamlFLAT/OFLAT systems.
- **Planning**: In this chapter we define and scheduled the tasks to be accomplished in this project.

2

Theory of Computation

In Theory of Computation, various models of computation with different levels of expressive power are studied. To discuss and compare different levels of expressive power, it is inevitable to consider algorithms in the context of some application domain. The typically chosen application domain is the syntax of formal languages. There are two techniques for defining languages that are typically used. The first technique is based on the concept of generation and uses the formalism of grammars. Another technique is based on the mechanism of recognition and uses the formalism of automata.

2.1 Chomsky Hierarchy

Noam Chomsky (with the help of Marcel-Paul Schützenberger) identified several classes of formal languages, with different levels of specification complexity, requiring computational models with different levels of expressive power to solve the problem of recognizing these languages [2].

The Chomsky hierarchy involves four levels of language types. Each type of language is associated with a type of generator(grammar) and a type of recognizer(automata).

- Type 0 — Unrestricted Grammars
- Type 1 — Context-Sensitive Grammars
- Type 2 — Context-Free Grammars
- Type 3 — Regular Grammars

2.1.1 Type 0: Unrestricted Grammars

Type 0 grammars, also known as unrestricted grammars, are the most general class in the Chomsky hierarchy. They have almost no restrictions on their production rules, meaning that any string of symbols can be replaced by any other string of symbols. This flexibility allows Type 0 grammars to generate any language that is recognizable by a Turing machine,

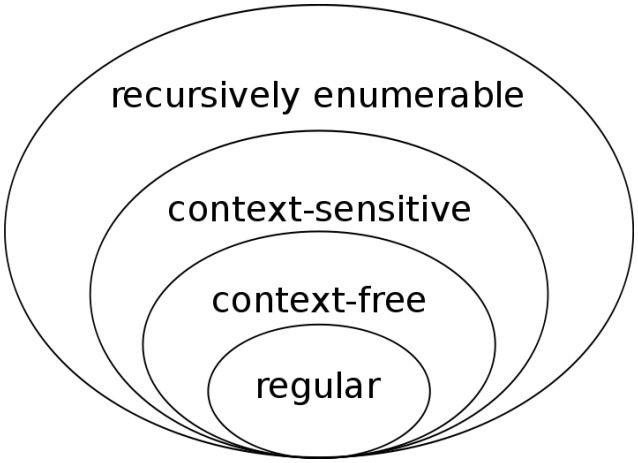


Figure 2.1: Chomsky hierarchy

making them equivalent in power to Turing machines. In other words, they can describe all recursively enumerable languages [2].

An example of an unrestricted grammar is:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow aAB, AB \rightarrow BA, A \rightarrow a, B \rightarrow b, B \rightarrow \lambda\}$$

Derivation of the word "aba":

$$\begin{aligned} S &\rightarrow aAB \\ aAB &\rightarrow aBA \\ aBA &\rightarrow abA \\ abA &\rightarrow aba \end{aligned}$$

2.1.2 Type 1: Monotonic Grammars

On type 1 grammars, or monotonic grammars, the length of the string on the left-hand side of a production rule is less than or equal to the length of the string on the right-hand side. This property ensures that the derivation process does not shorten the string. Type 1 grammars can generate languages that are recognized by a linear bounded automaton, which is a Turing machine with a tape of limited length proportional to the input size [2].

An example of a monotonic grammar is:

$$G = (\{S, B\}, \{a, b, c\}, P, S)$$

$$P = \{S \rightarrow aBSc, S \rightarrow abc, Ba \rightarrow aB, Bb \rightarrow bb\}$$

Derivation of the word "aaabbbccc":

$$
\begin{aligned}
S &\longrightarrow aBSc \\
aBSc &\longrightarrow aBaBScc \\
aBaBScc &\longrightarrow aBaBabccc \\
aBaBabccc &\longrightarrow aBaaBbccc \\
aBaaBbccc &\longrightarrow aaBaBbccc \\
aaBaBbccc &\longrightarrow aaaBBbccc \\
aaaBBbccc &\longrightarrow aaabbbccc
\end{aligned}
$$

2.1.3 Type 2: Context-Free Grammars

On type 2 grammars, or context-free grammars, the left-hand side consists of a single non-terminal symbol, and the right-hand side can be any string of terminal and/or non-terminal symbols. These grammars allows context-free grammars to generate languages that can be recognized by a pushdown automaton [2].

An example of a context-free grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \longrightarrow aSb, S \longrightarrow ab\}$$

Derivation of the word "aabb":

$$
\begin{aligned}
S &\longrightarrow aSb \\
aSb &\longrightarrow aaSbb \\
aaSbb &\longrightarrow aabb
\end{aligned}
$$

2.1.4 Type 3: Regular Grammars

On type 3 grammars, also known as regular grammars, represent the most constrained category within the Chomsky hierarchy. The production rules for these grammars are quite specific, each rule's left-hand side is a single non-terminal symbol, while the right-hand side is either a single terminal symbol or a terminal symbol followed by a single non-terminal symbol. This strict format allows regular grammars to generate languages that can be recognized by finite automata. [2].

An example of a regular grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \longrightarrow aS, S \longrightarrow b\}$$

Derivation of the word "aaab":

$$S \rightarrow aS$$
$$aS \rightarrow aaS$$
$$aaS \rightarrow aaaS$$
$$aaaS \rightarrow aaab$$

This dissertation will focus on a extension of type 2 grammars: Attribute Grammars.

2.2 Attribute Grammars

Attribute grammars are an extension of context-free grammars that associate attributes with the grammar symbols and define rules for computing these attributes, in addition, add conditions that are boolean expressions using the attributes and are used to restrict the generated language. These features provide a way to carry semantic information through the parse tree, enabling context-sensitive analysis [5].

There are two types of attributes:

- **Synthesized Attributes**: These are computed from the attributes of the children nodes in the parse tree. They propagate information up the tree.
- **Inherited Attributes**: These are computed from the attributes of the parent and/or sibling nodes. They propagate information down and across the tree.

By extending context-free grammars with attributes and condictions, attribute grammars enable more powerful and flexible language descriptions. In fact, it is easy to integrate contextual aspects into a grammar, often more intuitively than through a monotonic grammar. For example, for the language of sequences of 'a's with a length less than or equal to ten, instead of needing 10 rules, we can use an attribute and some condictions to reach the same result.

2.2.1 Synthesized Attributes

Synthesized attributes are computed from the attributes of the children of a node in the derivation tree. These attributes propagate information up the tree, allowing for the evaluation of properties that depend on the structure of the subtree rooted at a particular node [5].

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$
$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define a synthesized attribute length that calculates the length of the generated string. The rules for computing this attribute are as follows:

$$\text{length}(S) = \text{length}(A)$$

$$\text{length}(A \rightarrow aA) = 1 + \text{length}(A)$$

$$\text{length}(A \rightarrow b) = 1$$

Derivation of the word "aaab" along with the simultaneous computation of the attribute:

$$S \Rightarrow A, \quad \text{length}(S) = \text{length}(A)$$

$$A \Rightarrow aA, \quad \text{length}(A) = 1 + \text{length}(A)$$

$$aA \Rightarrow aaA, \quad \text{length}(A) = 1 + (1 + \text{length}(A))$$

$$aaA \Rightarrow aaaA, \quad \text{length}(A) = 1 + (1 + (1 + \text{length}(A)))$$

$$aaaA \Rightarrow aaab, \quad \text{length}(A) = 1$$

Back-substituting:

$$\text{length}(A) = 1$$
$$\text{length}(A) = 1 + 1 = 2$$
$$\text{length}(A) = 1 + 2 = 3$$
$$\text{length}(A) = 1 + 3 = 4$$

Thus, the length of the generated string **aaab** is:

$$\text{length}(S) = 4$$

2.2.2 Inherited Attributes

Inherited attributes are passed from parent nodes to child nodes in the derivation tree. These attributes propagate information down and across the parse tree [5].

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define an inherited attribute word that accumulates the word that is being generated. of the generated string. The rules for computing this attribute are as follows:

$$\text{word}(S) = ""$$

$$\text{word}(A \rightarrow aA) = \text{word}(A) + "a"$$

$$\text{word}(A \rightarrow b) = \text{word}(A) + "b"$$

Derivation of the word "aaab" along with the simultaneous computation of the attribute:

$$S \Rightarrow A, \quad \text{word}(S) = ""$$

$$A \Rightarrow aA, \quad \text{word}(A) = \text{word}(A) + "a"$$

$$aA \Rightarrow aaA, \quad \text{word}(A) = \text{word}(A) + "a"$$

$$aaA \Rightarrow aaaA, \quad \text{wordx}(A) = \text{word}(A) + "a"$$

$$aaaA \Rightarrow aaab, \quad \text{word}(A) = \text{word}(A) + "b"$$

Back-substituting:

$$\begin{aligned}
&\text{word}(S) = ""\\
&\text{word}(A) = \text{word}(S) + "a" = "a"\\
&\text{word}(A) = "a" + "a" = "aa"\\
&\text{word}(A) = "aa" + "a" = "aaa"\\
&\text{word}(A) = "aaa" + "b" = "aaab"
\end{aligned}$$

Thus, the accumulated word of the generated string is:

$$\text{word}(S) = "aaab"$$

2.2.3 Conditions

Consider the language of strings of the form $a^n b^n c^n$, where the number of 'a's, 'b's, and 'c's must be equal. A context-free grammar can generate strings with the correct structure but cannot enforce the equal lengths of the sequences [1]. But as we saw earlier, we can add conditions to an attribute grammar, ensuring this requirement is met. as we can see in the example below.

<letter_sequence> ::= <asequence> <bsequence> <csequence>

condition: Size(<asequence>) = Size(<bsequence>) = Size(<csequence>)

<asequence> ::= a

Size(<asequence>) ← 1

| <asequence> a

Size(<asequence>) ← Size(<asequence>) + 1

<bsequence> ::= b

Size(<bsequence>) ← 1

| <bsequence> b

Size(<bsequence>) ← Size(<bsequence>) + 1

<csequence> ::= c

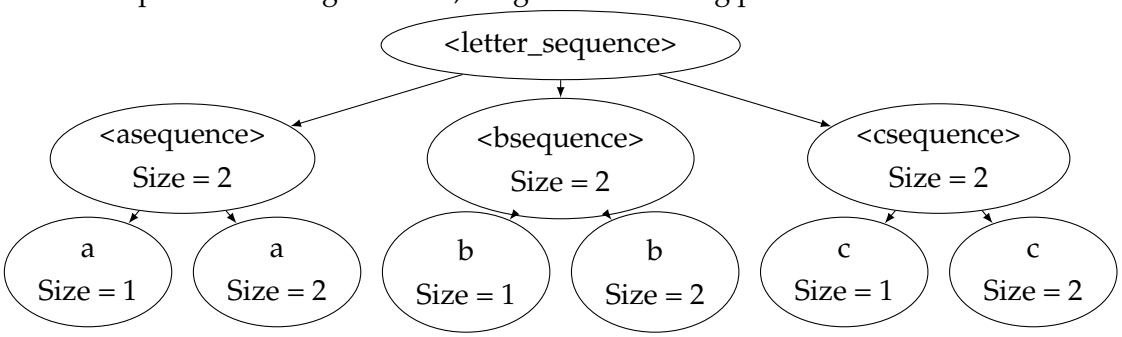
Size(<csequence>) ← 1

| <csequence> c

Size(<csequence>) ← Size(<csequence>) + 1

[1]

If we now parse the string "aabbcc", we get the following parse tree:



As we can see this tree shows the Size attribute is synthesized up the tree, ensuring that the length of the 'a's, 'b's, and 'c's sequences are equal. And then on the root we check the following condition:

Size(<asequence>) = Size(<bsequence>) = Size(<csequence>)

3 = 3 = 3

2.2.4 Circularity

For an attribute grammar to be well-formed, the attributes associated with nonterminals at any node in the parse tree must be evaluable based on the grammar's semantic rules

[4]. Since non-trivial context-free grammars can generate an infinite number of parse trees, determining well-formedness is a complex task [5]. The primary challenge lies in circular dependencies within the attribute dependency graph. For instance, if attribute a1 depends on a2, which in turn depends on a1, neither attribute can be evaluated, leading to an evaluation deadlock.

Consider the following grammar in figure 2.2

0: Number → Sign List	(i) List↓Scale := 0
	(ii) Number↑Value := IF Sign↑Neg THEN - List↑Value ELSE List↑Value
1: Sign → +	(i) Sign↑Neg := False
2: Sign → -	(i) Sign↑Neg := True
3: List → BinaryDigit	(i) BinaryDigit↓Scale := List↓Scale
	(ii) List↑Value := BinaryDigit↑Value
4: $List_0$ → $List_1$ BinaryDigit	(i) $List_1$↓Scale := $List_0$↓Scale + 1
	(ii) BinaryDigit↓Scale := $List_0$↓Scale
	(iii) $List_0$↑Value := $List_1$↑Value + BinaryDigit↑Value
5: BinaryDigit → 0	(i) BinaryDigit↑Value := 0
6: BinaryDigit → 1	(i) BinaryDigit↑Value := $2^{\text{BinaryDigit}\downarrow\text{Scale}}$

Figure 2.2: Attribute grammar for signed binary numbers [4]

The best way to visualize the circularity is to actually draw the dependency graph for the grammar. Figure 3 shows such a graph for the attribute grammar in figure 2.2.

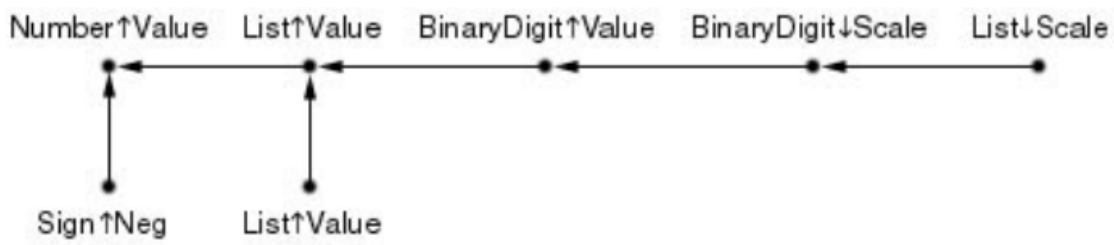


Figure 2.3: Attribute dependency graph for the grammar in figure 2 [4]

The dependency graph is constructed by combining the individual dependencies from each rule in the grammar, which explains the presence of multiple List↑Value attributes. In this case, the graph contains no circular dependencies, meaning it is well-formed—i.e., all possible parse trees generated by the grammar can be evaluated using the semantic rules. A circular dependency could be introduced by reversing the arrow from Sign↑Neg to Number↑Value and adding an arrow from Sign↑Neg to the lower List↑Value as we can see in the figure bellow. In such a scenario, none of the four attributes forming the cycle could be evaluated. The challenge in attribute grammars is detecting these cycles within the attribute dependency graph[4].

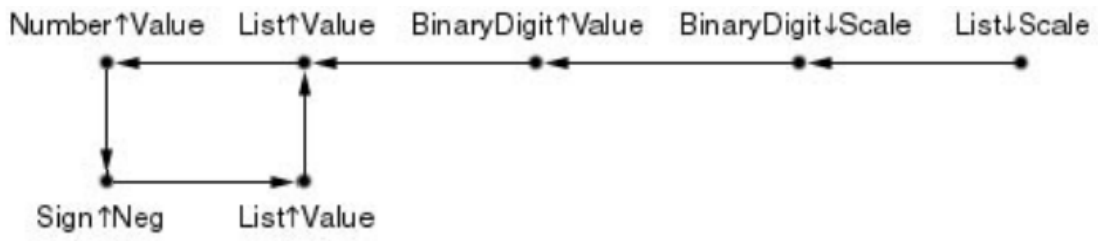


Figure 2.4: Modified attribute dependency graph with circularity

2.3 Automata

Automata theory is a fundamental area of theoretical computer science that deals with the study of abstract machines and the problems they can solve [10]. As described in the Chomsky hierarchy, different types of automata correspond to different classes of formal languages.

2.3.1 Types of Automata

2.3.1.1 Finite Automata

Finite automata (FA) are models in computer science used to recognize regular languages, they consist of a finite number of states, transitions between these states, and an input alphabet, when an input string is processed, the automaton transitions through states according to the input symbols. If it ends in an accepting state, the string is recognized as part of the language.

2.3.1.2 Pushdown Automata

Pushdown automata (PDA) are more powerful than finite automata and can recognize context-free languages. They extend finite automata by adding a stack, which provides additional memory. This stack allows PDAs to handle nested structures, such as parentheses in arithmetic expressions [6].

2.3.1.3 Turing Machines

Turing machines (TM) are the most powerful type of the Chomsky hierarchy and can recognize recursively enumerable languages. They consist of an infinite tape, a tape head that can read and write symbols, and a set of states. Turing machines can simulate any algorithm and are used to define the limits of what can be computed and are fundamental in the study of computability [6].

2.3.1.4 Linear Bounded Automata

Linear bounded automata (LBA) are a type of Turing machine with a tape of limited length proportional to the input size, they recognize context-sensitive languages. LBAs are more

powerful than PDAs but less powerful than general Turing machines, they also are used in certain types of language recognition and computational complexity studies [6].

3

Grammar Tools

The tools for FLAT aimed at automata can be classified into two major categories: text-based automaton simulators and graphical interface-based automaton simulators. The first involves specifying a model through structured text, which must be processed using a parser, the second uses a graphical interface that allows creating, editing, and using an automaton to recognize words

The different existing tools can also support only one type of automaton or multiple types of automata, such as finite automata, pushdown automata, and Turing machines. There are also tools that deal with regular expressions and grammars, and in such cases, it is common for conversion operations to be available between automata and vice versa.

Next, some pedagogical tools are presented and classified according to the indicators described above, i.e., whether they are text-based tools or graphical interface-based tools; whether they implement only one type of automaton or multiple types, and whether they support regular expressions and grammars. We will discuss three such tools: **JFLAT**, **ANTLR**, and **CGM**.

3.1 JFLAT

JFLAT (Java Formal Languages and Automata Toolkit) is a versatile tool designed for experimenting with formal languages, it supports various computational models, including finite automata, pushdown automata, Turing machines, and different types of grammars this tool also provides a comprehensive environment for users to explore and understand the theoretical foundations of computer science. JFLAT offers several key features that enhance its utility as an educational tool as it allows users to create and visualize the computational models helping students better understand the structure and behaviour of the same models. This tool supports the conversion between different forms of automata. Additionally, JFLAT includes parsing capabilities for various types of grammars, enabling users to experiment with context-free grammars and other formal language constructs.

JFLAT also provides educational benefits by offering a visual and interactive approach to learning complex concepts, as it have the ability to visualize automata and grammars

helping users to gain a deeper understanding of theoretical concepts through hands-on experimentation, develop problem-solving skills by working through practical exercises and projects. Examples of exercises and projects include creating and testing finite automata to recognize specific patterns or languages, designing and parsing context-free grammars to explore the syntax and structure of programming languages.

3.2 ANTLR

ANTLR (ANother Tool for Language Recognition) is a parser generator used for reading, processing, and translating structured text or binary files, it offers several notable features, including the ability to generate parsers, lexers, and tree parsers from grammars, facilitating the creation of language processors. It also supports multiple programming languages, making it versatile for various development environments and also can be used to build languages, interpreters, compilers, and other language-related frameworks.

This tool help users understand the principles of syntax analysis and parsing, building parsers and compilers, and explore the language design and implementation. ANTLR can be used in various educational projects and assignments, such as creating parsers for custom programming languages, building simple compilers to learn about the different stages of compilation, and developing domain-specific languages to appreciate the practical applications of the tool in real-world scenarios.

3.3 CGM: Context-Free Grammar Manipulator

CGM (Context-Free Grammar Manipulator) is a tool developed at the University of Porto designed to assist in the manipulation and study of context-free grammars. It provides a graphical environment for editing grammars, converting them to Chomsky normal form, parsing words, and constructing parse trees interactively, it offers several key functionalities, including grammar editing, Chomsky normal form conversion, and the ability to parse words and visualize the corresponding parse trees, aiding in the understanding of grammar structures.

CGM helps users to visualize the structure and operation of context-free grammars, experiment with different grammar configurations to see how changes affect parsing and parse tree construction, and develop a deeper understanding of the concepts of grammar rules, parsing, and syntax trees.

4

OFLAT/OCaml-Flat

4.1 OCamlFLAT Overview

This chapter introduces the OFLAT/OCamlFLAT system in its current state. The system comprises two main components: the OFLAT web application, which provides a graphical user interface, and OCamlFLAT, the underlying library that handles the system's logic, including all FLAT models and their representations.

OFLAT is an ongoing development at the NOVA-LINCS research center within the Computer Science Department of FCT-UNL, supported by two projects: Factor (Functional ApproaCh Teaching pOrtuguese couRses) and LEAFs [9]. It is implemented in OCaml, utilizing the `js_of_ocaml` and Cytoscape.js libraries. Additionally, it leverages the OCamlFLAT library for logical operations. A screenshot of the OFLAT platform's user interface is shown in Figure 4.1

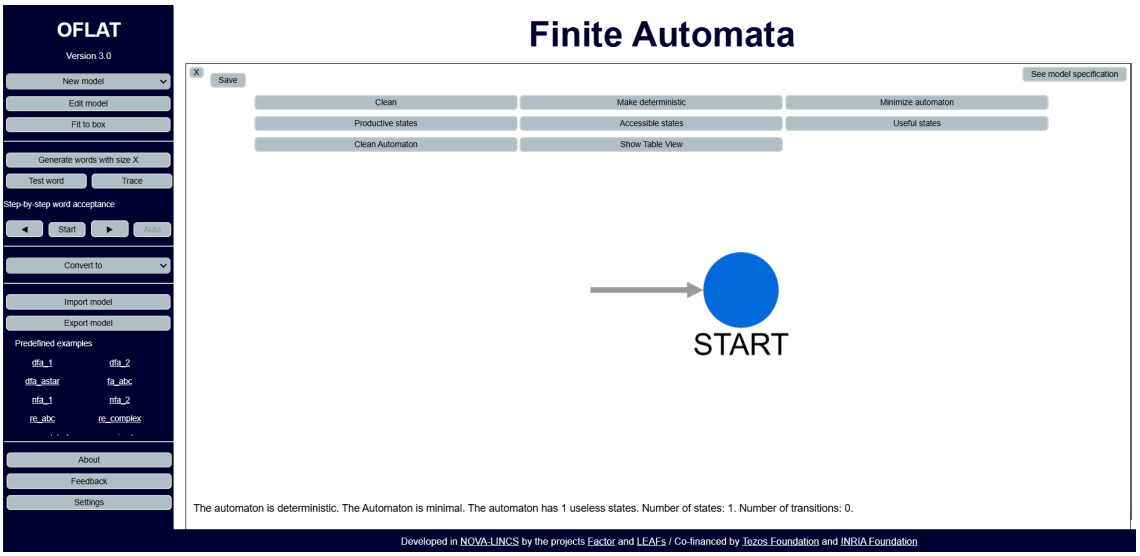


Figure 4.1: Current Version OFLAT User Interface

It supports working with a wide range of previously introduced language models, covering all types defined by Chomsky. This tool serves as a valuable resource for students

as a study aid and for educators as a teaching tool worldwide. Its intuitive graphical interface distinguishes it from most other tools discussed in this document, many of which are outdated and lack modern or functional graphical interfaces.

This section provides a detailed examination of the internal workings of OFLAT and the libraries it relies on.

4.2 OFLAT User Interface

The OFLAT User Interface (UI) is primarily divided into two main sections: the workspace and the sidebar (as shown in Figure 4.2).

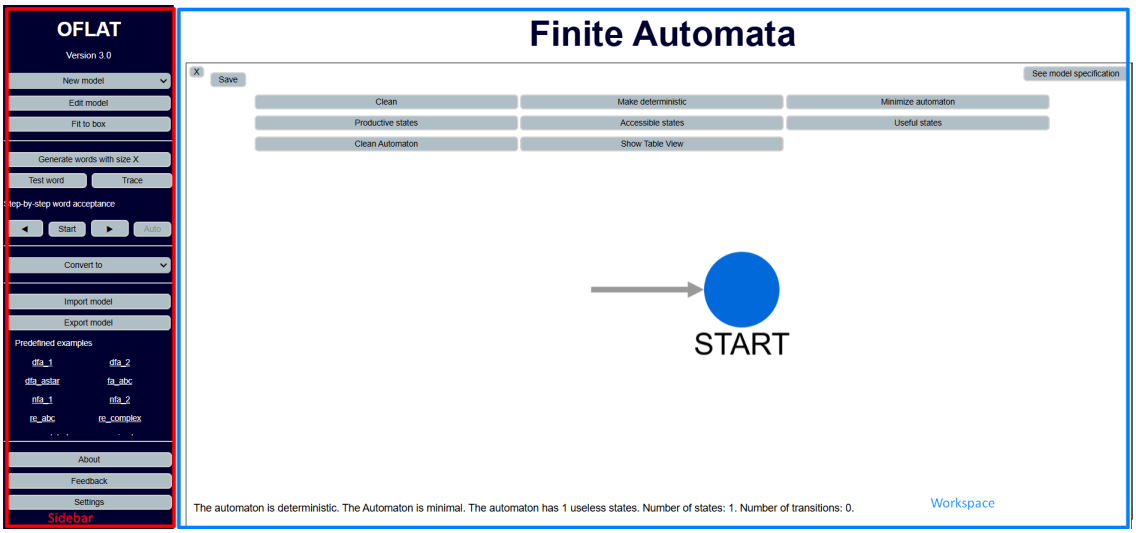


Figure 4.2: OFLAT User Interface Division. Red - Sidebar, Blue - Workspace

The sidebar contains buttons for creating and editing models within the workspace. Additionally, it provides access to key operations such as conversions and word acceptance. Alongside the About, Feedback, Language, and Settings buttons, there is also an Examples tab, which features a scrollable list of exercises and examples curated by the OFLAT development team. These examples allow users to test their knowledge or learn through guided practice.

The workspace section serves as the main area where users can visualize and interact with their chosen models. Depending on the selected model, a set of relevant buttons appears at the top of the workspace, offering access to specific operations. The workspace layout is illustrated in Figure 4.3.

However, when displaying a model specification or visualizing an exercise, the workspace splits into two sections. The model remains on the left side, while a window opens on the right, displaying the specification or exercise, creating the previously mentioned split (as shown in Figure 4.4). This layout allows users to view both elements simultaneously, making it easier to correlate them with minimal effort.

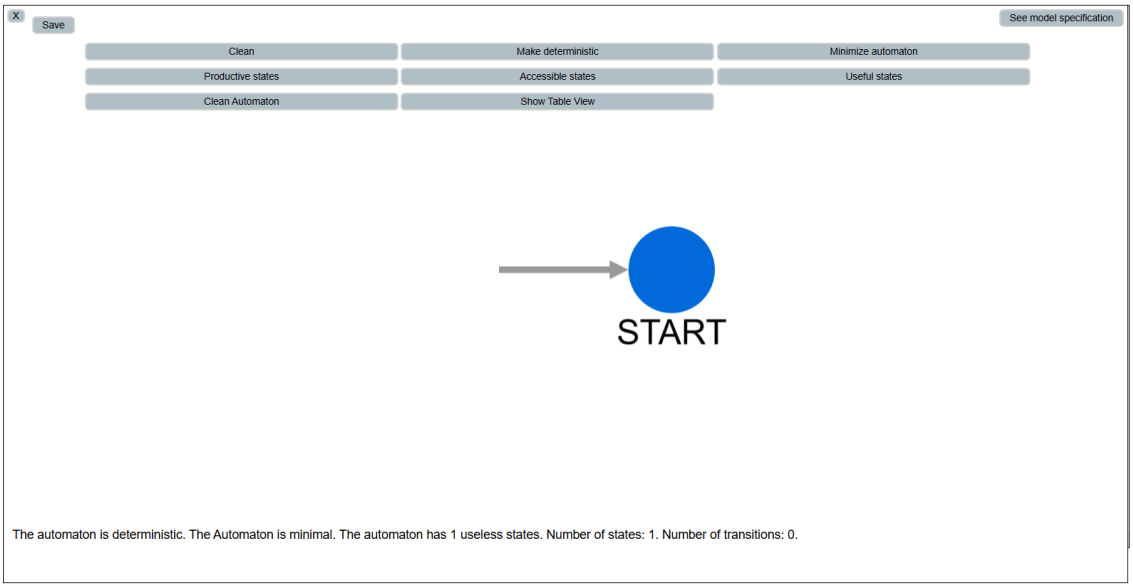


Figure 4.3: Workspace Closeup

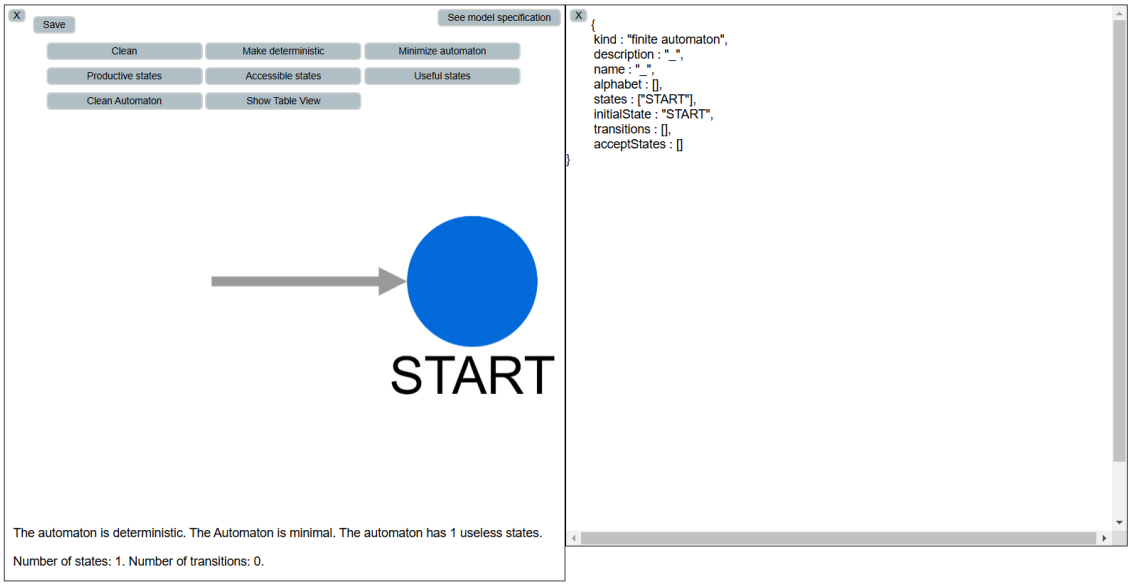


Figure 4.4: Workspace Window Split

This feature is particularly beneficial for exercises where step-by-step visualization—such as word acceptance—helps users identify and correct potential errors in their input model.

5

Technology

5.1 Programming with OCaml

OCaml is a high-performance, portable functional programming language developed in 1996 by Xavier Leroy, Jérôme Vouillon, Damien Doligez, and Didier Rémy at INRIA in France [8]. Like other functional programming languages, OCaml is built around the concept of expressions, meaning computations are performed by evaluating expressions, which in turn produce values.

The example bellow, shows a few examples of OCaml computations:

```
  (Definition of the binary tree type)
  # type tree = Node of int * tree * tree | Nil;;
  - : type tree = Node of int * tree * tree | Nil


  (Definition of a variable that contains a tree)
  # let t : tree = Node(1, Nil, Node(2, Nil, Nil));;
  - : val t : tree = Node(1, Nil, Node(2, Nil, Nil));;

  (Creation of a function that calculates the height of a tree)
  # let rec height t =
    match t with
    | Nil -> 0
    | Node(_,l,r) ->
          1 + max (height l) (height r)
  ;;
- : val height : 'tree -> int *
```

5.1.1 Declarative Programming

In functional programming languages like OCaml, the code is often written in a declarative style. Declarative programming focus in describing the logic of a computation rather than controlling its flow, this approach offers several advantages, for example by abstracting away the flow of control, the code becomes simpler compared to imperative programming. Additionally, since the code aligns more closely with the underlying concepts of the project, it tends to be more understandable and less prone to logical errors. Although this may come at the cost of efficiency, it benefits the OFLAT platform and OCamlFlat library, where the theoretical concepts involved are complex.

5.2 OCaml/JavaScript Interoperability

Interoperability between OCaml and JavaScript is made possible through the use of the `js_of_ocaml` toll [3] , which translates OCaml code into JavaScript, allowing OCaml code to run in a browser.

This tool consists of three main components:

- `js_of_ocaml` ppx: A preprocessor that extends the OCaml library, adding functionalities that facilitate integration with JavaScript.
- `js_of_ocaml` library: A library that contains bindings for various categories of JavaScript objects, enabling OCaml code to efficiently interact with JavaScript APIs and libraries.
- `js_of_ocaml` compiler: A compiler that translates OCaml code into JavaScript, allowing OCaml programs to be executed in web-based environments.

5.2.1 JavaScript Types

Types in OCaml and JavaScript are not represented in the same way in both languages. For example, a string in OCaml is a mutable array of bytes, whereas in JavaScript, strings are constant arrays of UTF-16 code points. Therefore, it is necessary to find a way to represent JavaScript types in OCaml appropriately. The table below illustrates the correspondences between the types in the two languages.

To ensure effective interoperability, it is crucial to understand how each type is handled in OCaml and JavaScript. For instance, besides strings, other types such as numbers, arrays, and objects also have distinct representations. In OCaml, integers and floating-point numbers are treated differently compared to JavaScript, where all numbers are essentially floating-point. Arrays in OCaml are mutable, while in JavaScript, arrays can be either mutable or immutable, depending on the context.

5.2.2 Bindings

Bindings are present in both the libraries within the *js_of_ocaml* environment and the OFLAT code itself. They allow the connection between two different programming languages, enabling the use of a library foreign to one of them.

Primarily, *js_of_ocaml* manages bindings by assigning JavaScript objects a distinct OCaml object type. This type is defined as `<m₁ : t₁; ...; mₙ : tₙ> Js.t`, where m_i represents a field of the JavaScript object and t_i its corresponding type.

5.2.3 `js_of_ocaml` Modules

The interoperability between OCaml and JavaScript is handled by the *js_of_ocaml* compiler in different ways. Within the compiler, the `Js` module exists to manipulate JavaScript values (such as strings, booleans, null, and undefined values) from OCaml and to perform conversions between the two languages. For example, the type `'a Js.opt` represents values that are either `null` or of type `'a`.

Additionally, this module includes another important module: `Js.Unsafe`. It provides a collection of unsafe JavaScript operations, ranging from type coercion to setting, getting, and deleting object properties. These operations are very powerful and should ideally be used sparingly across the project. However, in some cases, their use may be necessary. By creating bindings and ensuring proper typing, most of these calls can be avoided.

The `Js` module is one of several modules included in the libraries that come with the compiler. Other key modules worth mentioning are `Dom`, which serves as a partial binding to the DOM Core API; `Dom_html`, which acts as a partial binding to the DOM HTML API; and the `Firebug` module, which provides a debugging console for development.

5.3 Cytoscape.js

Cytoscape.js is an open-source graph theory library written in JavaScript that enables the manipulation of graphs. It functions as a standalone JavaScript library and utilizes HTML5 canvases for rendering its visualization components.

In OFLAT, Cytoscape.js is primarily used for the graphical representation of automata through nodes and edges. Additionally, it is notably used whenever OFLAT needs to display a tree (important in the scope of context-free grammars).

With this library, users can perform various operations on the displayed automaton, such as creating new states and transitions, allowing for interactive manipulation of the model.

6

Planning

6.1 Introduction

In this chapter, the work plan for this dissertation is presented, considering a period of 24 weeks (6 months).

The objective of this work is to add support for attribute grammars to the system OCamlFLAT/OFLAT. The implementation work aims to extend existing software, which already includes some relevant elements, such as support for context-free grammars and basic visualization and graphical interaction mechanisms.

6.2 Support for Attribute Grammars in the OCamlFLAT Library (8 weeks)

At the library level, several functionalities are planned:

- Representation of attributes and their calculation rules within grammars.
- Circularity testing of synthesized and inherited attributes in a grammar.
- Calculation of synthesized and inherited attributes on an already constructed derivation tree.
- Calculation of synthesized and inherited attributes during the derivation tree construction of a word, i.e., during parsing.
- Introduction of "conditions" in grammars to use attribute values to restrict the generated language.
- Integration of attribute grammars into the existing support for composite models in the library. This involves attempting various operations on attribute grammars: union, intersection, difference, concatenation, and Kleene closure.
- Integration of grammars into the existing support for pedagogical exercises in the library.

6.3 Support for Grammars in the OFLAT Web Application (6 weeks)

Attribute grammars must be integrated into the existing graphical interface. The additions should have pedagogical features, and some interesting ideas may emerge for the graphical interface.

There is already support for context-free grammars in the graphical application. That part of the code will be duplicated, refactored, and adapted. Understanding and modifying the graphical application code is not a trivial task.

What should be supported in the graphical interface for attribute grammars:

- Visualization.
- Editing.
- Visualization of the results of operations available in the library.
- Visualization of derivation trees annotated with attribute calculations. In a more ambitious version, attribute calculation could be animated.
- Integration into the existing support for composite models in the graphical application.

6.4 Pedagogical Exercises (1 week)

Create a collection of interesting pedagogical examples and encode them into a set of exercises in the format supported by the OCamlFLAT/OFLAT system.

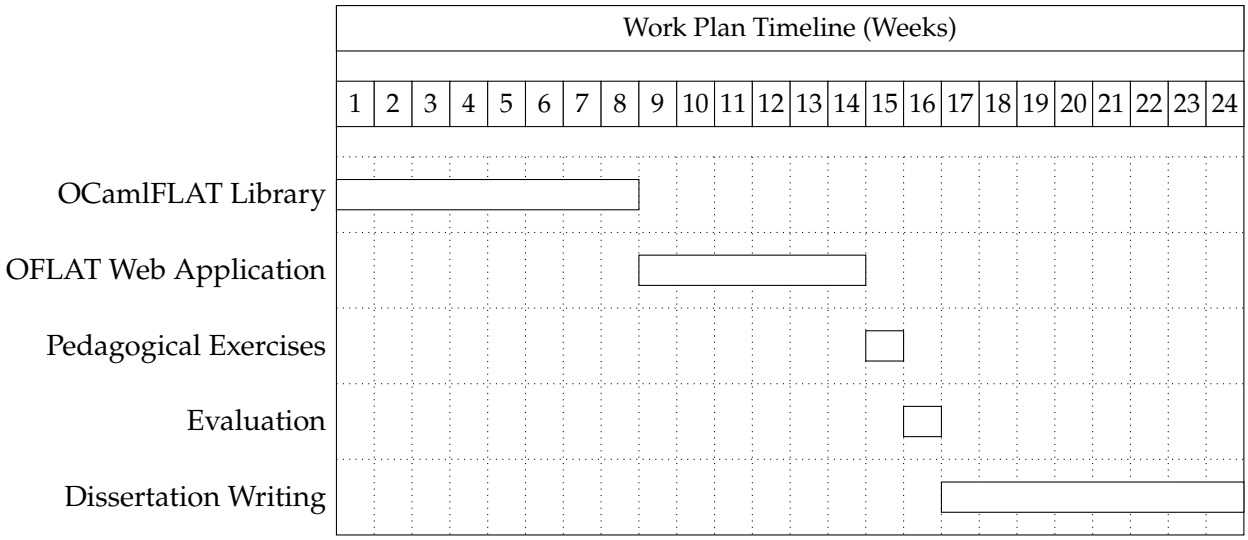
6.5 Evaluation (1 week)

- The correctness of the newly implemented code will be assessed through a set of unit tests, which will be written alongside code development.
- Analyzing the system's limitations by creating examples that pose challenges due to size or execution time.
- Seeking feedback from potential users of the system, particularly regarding the graphical interface. For example, teachers and students of the TC course.

6.6 Dissertation Writing (8 weeks)

Ideally conducted in small increments, concurrently with other tasks. Two months are considered for the production of the final document. It is important to have a near-final version ready two weeks before the submission deadline to facilitate inviting an examiner.

6.7 Gantt Chart



Bibliography

[1] *Chapter 3 ATTRIBUTE GRAMMARS*. Accessed: 2025-01-30. URL: https://homepage.divms.uiowa.edu/~slonnegr/plf/Book/Chapter3.pdf (cit. on pp. 8, 9).

[2] N. Chomsky. "Three models for the description of language". In: *IRE Transactions on Electronic Computers* EC-8.1 (1956) (cit. on pp. 3–5).

[3] B. Demange et al. *From Bytecode to JavaScript: The Js of ocaml Compiler*. 2012. URL: https://citeseerx.ist.psu.edu/document?doi=1b87d68c7c4f7b897eeb098046 57225f6f8f762f&repid=rep1&type=pdf (cit. on p. 20).

[4] C. Gordon. *The Circularity Problem for Attribute Grammars*. Accessed: 2025-01-17. 2000. URL: https://courses.cs.washington.edu/courses/cse531/00au/gordon.pdf (cit. on p. 10).

[5] D. E. Knuth. "Semantics of Context-Free Languages". In: *Theory of Computing Systems* 2.2 (1968), pp. 127–145 (cit. on pp. 6, 7, 10).

[6] D. E. Knuth. "Semantics of Context-Free Languages". In: *Theory of Computing Systems* 2.2 (1968), pp. 127–145 (cit. on pp. 11, 12).

[7] J. M. Lourenço. *The NOVAthesis LaTeX Template User's Manual*. NOVA University Lisbon. 2021. URL: https://github.com/joaomlourenco/novathesis/raw/main/template.pdf (cit. on p. i).

[8] *OCaml History*. Accessed: 2025-01-17. URL: https://ocaml.org/about (cit. on p. 19).

[9] *OFLAT*. Accessed: 2025-01-30. URL: http://ctp.di.fct.unl.pt/FACTOR/OFLAT/ (cit. on p. 15).

[10] A. Ravara. *Slides da disciplina de teoria da computação*. Rel. téc. FCT-UNL, 2020 (cit. on p. 11).